okay give me the comprehensive, logic or algorithm architecture for this from start to finish, what I need to do

Yes — the right architecture is a **multi-key matching engine** with explicit match modes, not a single normalization rule. On multilingual sites, different language or regional URLs should be treated as different URLs by default, and Adobe-style reporting can vary between full URL, short URL, and path-based dimensions, so your parser has to detect the export type first before it decides how to match.[1] [2]

Core principle

Your app needs to separate **row identity** from **page identity**. Adobe reporting can expose a full page URL or a shorter URL-style value, and query-string handling can also vary depending on implementation settings, so one "normalize everything" rule will create false matches.[3] [2]

The safest industry-style rule is: **exact first, normalized second, page-rollup third, suggestion last.** RFC 3986 allows controlled normalization such as scheme/host handling and percent-encoding equivalence, but it does not justify dropping meaningful path segments like `/se/en`, `/us/en`, or `/cn/zh` during generic comparison.

Import pipeline

When a file is uploaded, the first job is to classify the export into one of these profiles:

- `FULL_URL_EXPORT`
- `FULL_URL_WITH_QUERY_EXPORT`
- `HOST_AND_PATH_EXPORT`
- `PATH_ONLY_EXPORT`
- `UNKNOWN_EXPORT`

That classification matters because a path-only report and a full-URL report are not the same dimension, and analytics tools often distinguish between full page URL and shorter URL/path style values. Adobe also supports including query strings in URL evaluation when configured, which means the same page may appear with or without parameters depending on how the export was built.[4] [2] [3]

After classification, each row should be parsed into a normalized internal record:

- `raw_value`

- `detected_profile`
- `host`
- `path`
- `query`
- `fragment`
- `locale_prefix`
- `raw_exact_key`
- `normalized_url_key`
- `page_identity_key`
- `path_key`
- `canonical_path_key`
- `match_type`
- `match_confidence`
- `is_ambiguous`

Key generation

For every row, generate these comparison keys:

- `raw_exact_key`: exact exported string, unchanged.
- `normalized_url_key`: lowercase scheme/host, strip default port, normalize safe percent-encoding, preserve the full path, preserve locale segments, optionally remove fragment for compare.
- `path_key`: path only, exactly as represented.
- `page_identity_key`: same as normalized URL but with **known tracking parameters** removed, such as `utm_*`, `gclid`, `fbclid`, and `source`; keep functional parameters separate.
- `canonical_path_key`: only created if you have an explicit mapping rule for canonicalized exports.

For multilingual URLs, keep locale or market segments as part of identity by default. Google's multilingual guidance recommends different URLs for different language versions, and canonical/hreflang handling is explicit rather than a reason to silently merge those URLs in analytics matching.[1]

Matching algorithm

Use this match order:

1. **Exact raw match**
2. **Exact normalized URL match**
3. **Exact page identity match**
4. **Exact path match**

5. **Exact canonical-path match, only if explicit mapping exists**
6. **Suggestion-only similar results**

Here is the behavior you want:

- If the query URL is full and the export is full, require full equality first.
- If the export is path-only, compare against the query's derived `path_key`.
- If the export is canonicalized, compare only against `canonical_path_key`, and label the result clearly as canonical, not exact.
- If multiple rows match a weaker key, return `ambiguous_match` instead of auto-selecting one.

That approach prevents `/se/en/...` from incorrectly matching `/us/en/...` or the non-locale version as an "exact" result, because those are different paths and different localized URLs by default. Partial-path logic can be useful for redirect or catch-all style systems, but it only matches paths and should be treated as fallback behavior that must be tested carefully, not as your truth layer for analytics identity.[5] [1]

Parameter rules

You need three parameter buckets:

- **Tracking parameters**: `utm_*`, `gclid`, `fbclid`, `msclkid`, `source`
- **Functional parameters**: filters, pagination, search state, language toggles
- **Unknown parameters**: keep until classified

Apply them like this:

- `raw_exact_key` keeps everything.
- `normalized_url_key` keeps everything except optional fragment handling.
- `page_identity_key` removes only tracking parameters.
- Functional parameters stay if they change the actual page state, such as filters or result sets.

This is important because Adobe URL handling can include query strings depending on configuration, so parameters may be part of the recorded value rather than noise. At the same time, enterprise reporting often benefits from separating clean page identity from campaign-specific URL variants, especially when full URL values explode cardinality.[6] [2] [3]

Fragment rules

Store the fragment in the raw record, but do not trust it as the only comparison key. Adobe community discussions show that hash-based URLs may not always appear in reports the same way users expect, so the safest design is to preserve the raw value and also create a no-fragment comparison key.[7]

Practical rule:

- Raw storage: keep fragment if exported.
- Exact raw match: compare fragment only when both sides include it.
- Normalized/page identity match: usually ignore fragment unless your export profile explicitly depends on it.

Locale logic

Locale segments must be explicit and preserved:

- `/se/en/...` is different from `/us/en/...`
- `.com` is different from `.cn`
- `/ch/de/...` is different from `/de/de/...`

Do not strip country/language segments unless the import profile says the export is already canonicalized to a locale-neutral path. Multilingual SEO guidance treats different language versions as different URLs, and canonical rules are deliberate mappings, not permission for generic collapsing.[1]

Confidence model

Use a simple confidence system:

- `1.00` = raw exact
- `0.95` = normalized exact
- `0.85` = page identity
- `0.70` = exact path only
- `0.55` = canonical-path mapping
- `0.20` = suggestion only

Do not use confidence to override logic. Confidence is for display and debugging; the match type decides whether the result is valid, ambiguous, or suggestion-only.

Data model

Use three main tables or collections:

- `imports`
  - `import_id`
  - `file_name`
  - `export_profile`
  - `imported_at`
- `rows`
  - `row_id`
  - `import_id`

    - `raw_value`
    - `host`
    - `path`
    - `query_json`
    - `fragment`
    - `raw_exact_key`
    - `normalized_url_key`
    - `page_identity_key`
    - `path_key`
    - `canonical_path_key`
- `canonical_mappings`
    - `mapping_id`
    - `source_pattern`
    - `target_canonical_path`
    - `rule_type`
    - `active`

This lets you keep exact analytics truth, while still supporting future rollups and reporting automation.

End-to-end flow

From start to finish, the app should do this:

1. Upload file.
2. Detect file type and parse rows.
3. Infer export profile from row samples.
4. Generate all keys for every row.
5. Save rows into the local store.
6. User enters a query URL or path.
7. Parse the query into the same key set.
8. Match using profile-aware priority rules.
9. Return:

- matched value
- match type
- confidence
- source row count
- warnings

- suggested alternatives if needed

Warnings should include:

- “Matched by canonical path, not exact URL.”
- “Matched by path-only export.”
- “Multiple variants found across locales.”
- “Tracking parameters removed for page identity match.”

Pseudocode

```
function matchQuery(query, importProfile, rows):
  q = buildKeys(query)

  if importProfile == FULL_URL_EXPORT:
    if exact(rows.raw_exact_key, q.raw_exact_key): return RAW_EXACT
    if exact(rows.normalized_url_key, q.normalized_url_key): return NORMALIZED_EXACT
    if exact(rows.page_identity_key, q.page_identity_key): return PAGE_IDENTITY
    return NO_MATCH_WITH_SUGGESTIONS

  if importProfile == PATH_ONLY_EXPORT:
    if exact(rows.path_key, q.path_key): return EXACT_PATH
    if exact(rows.canonical_path_key, q.canonical_path_key): return CANONICAL_PATH
    return NO_MATCH_WITH_SUGGESTIONS

  if importProfile == UNKNOWN_EXPORT:
    try RAW_EXACT
    try NORMALIZED_EXACT
    try PAGE_IDENTITY
    try EXACT_PATH
    try CANONICAL_PATH
    if more_than_one_match: return AMBIGUOUS
    return NO_MATCH_WITH_SUGGESTIONS
```

What you need to do now

For v1, build in this order:

1. **Define export profiles**
   Decide the row formats you want to support first: full URL, full URL with query, host+path, and path only.
2. **Build the URL parser**
   Create one shared parser that can take either a full URL or a path and produce all keys consistently.
3. **Implement safe normalization only**
   Lowercase scheme/host, preserve path, preserve locale, classify parameters, and create comparison keys.[1]
4. **Add the profile-aware matcher**
   Do not let path-only rules run against full-URL identity unless the export type requires it.

5. **Add ambiguity handling**
   If two or more results qualify at the same priority level, stop and show ambiguity instead of choosing one.
6. **Add debug visibility in UI**
   Always show:

- input query
- export profile
- chosen match type
- confidence
- matched row
- discarded variants

7. **Add canonical mapping as a separate feature**
   Do not hardcode locale stripping into the normalizer. Make it a deliberate rules layer.
8. **Test with real Hitachi Energy samples**
   Use cases should include:

- same path, different locale
- same page, different tracking params
- path-only export
- full URL export
- fragment URL
- `.com` vs `.cn`

Final rule

If you want one line to guide the whole app, use this:

Preserve exact URL truth first, then create controlled reporting identities on top of it. That is the safest approach for multilingual analytics data, mixed export types, and future automation.[2] [1]

I can next turn this into a clean spec you can paste directly into your project docs, including Rust-oriented function breakdowns and exact matcher definitions.

⁂

1. https://developers.google.com/search/docs/specialty/international/managing-multi-regional-sites
2. https://www.linkedin.com/pulse/must-have-page-url-report-adobe-analytics-andrey-osadchuk
3. https://experienceleague.adobe.com/en/docs/experience-manager-65/content/sites/administering/integration/adobeanalytics-link
4. https://www.analyticsmania.com/post/full-page-urls-in-google-analytics-4/
5. https://help.urllo.com/en/articles/6319448-understanding-partial-path-matching

6. https://www.rohankapooronline.com/2018/09/adobe-analytics-reporting-issues-and.html
7. https://experienceleaguecommunities.adobe.com/adobe-analytics-3/page-urls-in-adobe-analytics-94445
8. https://experienceleague.adobe.com/en/docs/analytics/components/marketing-channels/mchannel-best-practices
9. https://stackoverflow.com/questions/29983246/matching-partial-url-regex-google-analytics
10. https://experienceleaguecommunities.adobe.com/adobe-analytics-3/full-url-84694
11. https://blog.laratranslate.com/website-localization-best-practices/
12. https://experienceleague.adobe.com/en/docs/experience-platform/destinations/ui/activate/export-datasets
13. https://poeditor.com/blog/multilingual-url-best-practices/
14. https://experienceleague.adobe.com/en/perspectives/from-data-to-insights-navigating-adobe-analytics-feeds
15. https://www.youtube.com/watch?v=oqkv9keDcTk
16. https://simplelocalize.io/blog/posts/urls-in-website-localization/